

COMP 3069

Computer Graphics

Lecture 9: Textures

Autumn 2024



The University of
Nottingham

UNITED KINGDOM · CHINA · MALAYSIA

0

Contents

- ◆ Texture, texel, UV, texture mapping, and texture aliasing
- ◆ Texture sampling: UVs, Projector, Corresponder, Sample and Transforms
- ◆ Magnified texture with and without bilinear filtering
- ◆ Texture differentials calculated and used
- ◆ Mipmap and its use
- ◆ Minified texture with and without mipmaps and trilinear filtering



The University of
Nottingham

UNITED KINGDOM · CHINA · MALAYSIA

1

1

Texture Mapping 纹理映射

- ◆ Texture mapping involves sampling a texture for each fragment on a triangle

纹理映射涉及为三角形上的每个片段采样纹理。

意思是：

当三角形被 rasterization（光栅化）之后，会产生很多 fragment，每个 fragment 都要根据自己的 UV 坐标，从纹理图里找颜色。

下方那张三角形图片，就是示例：

三角形本身只是一个平面形状，纹理图是一张真实的照片通过 UV 映射后，照片的内容被“贴”到三角形上。



Texture

- ◆ A texture is an image



Texel

- ◆ A **texel** is a texture element, which is essentially a colour 纹理元素（简称“纹素”）本质上是一种颜色。



UV

- ◆ A UV is a **location** on a texture. In this figure, three UVs are drawn with red points UV 是纹理上的一个位置。在这幅图中，三个 UV 用红色点标出。

UV 是纹理空间 (Texture Space) 里的坐标，范围通常是 (0, 1)。U 对应 纹理的 x 方向 (横向) V 对应 纹理的 y 方向 (纵向)



可以把 UV 想象成“百分比位置”：

- (0, 0) = 左下角
- (1, 0) = 右下角
- (0, 1) = 左上角
- (1, 1) = 右上角
- (0.5, 0.5) = 图像正中心

所以 UV 不是像素坐标，而是一个归一化 (normalized) 的位置。

(X, Y, Z)

fragment 的三维坐标

↓ PROJECT

(u, v)

对应纹理图中的位置 (归一化)

↓ CORRESPOND

(Tx, Ty)

纹理图像素坐标 (浮点数, 可含小数)

↓ SAMPLE

(r, g, b)

读取到的 texel 原始颜色

↓ TRANSFORM

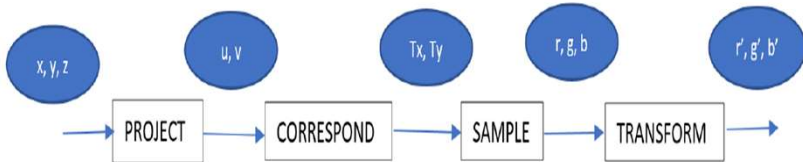
(r', g', b')

最终 fragment 显示颜色

Texture Sampling

保护 对应 样本

- ◆ **Four steps:** Project, Correspond, Sample, Transform 转换

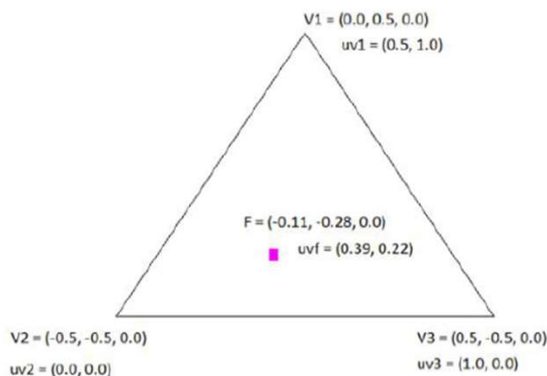


Specifying UV

```
float vertices[] =  
{  
    //pos                //UV  
    0.f, .5f, 0.f,        .5f, 1.f  
    -.5f, -.5f, 0.f,      0.f, 0.f,  
    .5f, -.5f, 0.f,       1.f, 0.f  
}
```

Projection to UV

- Texture coordinate (UV) of a fragment can be interpolated from the vertices
- 三角形内部的任意 fragment 的 UV，可以用三角形三个顶点的 UV 做插值得到。



令 F 在三角形内部写成三个顶点的线性组合：

$$F = \alpha V_1 + \beta V_2 + \gamma V_3$$

并且：

$$\alpha + \beta + \gamma = 1$$

这三个系数 α, β, γ 就是重心坐标，代表 F 离三个顶点的“权重”。

UV 也按同样的权重插值：

$$uv_F = \alpha uv_1 + \beta uv_2 + \gamma uv_3$$

u 分量：

$$u_F = 0.22 \cdot 0.5 + 0.5 \cdot 0 + 0.28 \cdot 1.0 = 0.11 + 0 + 0.28 = 0.39$$

v 分量：

$$v_F = 0.22 \cdot 1.0 + 0.5 \cdot 0 + 0.28 \cdot 0 = 0.22$$

所以：

$$uv_F = (0.39, 0.22) \quad \checkmark$$

8

Corresponder

- A corresponder function maps the fragment UV to a texel coordinate, e.g., $(0.39, 0.22) \rightarrow (390.00, 146.52)$

Corresponder = 把 UV (0~1) 转成纹理像素坐标 (Tx, Ty)。



✨ 1. 为什么要从 UV 转成 texel 坐标？

因为：

- UV 范围是 (0~1) → 这是归一化坐标
- 纹理是一个像素矩阵
- 像素位置必须是实际尺寸相关的坐标

如果纹理是 1000×665 像素，那么：

- $u = 0.39 \rightarrow$ 对应横向 39% $\rightarrow 39\% \times 1000 = 390$ 像素
- $v = 0.22 \rightarrow$ 对应纵向 22% $\rightarrow 22\% \times 665 = 146.52$ 像素

假设纹理大小为：

- width = 1000
- height = 665

fragment 的 UV：

- UV = (0.39, 0.22)

★ 对应公式：

$$T_x = u \times width$$

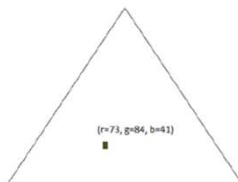
$$T_y = v \times height$$

9

Sampling & Transform

Sampling (采样) : 丢弃小数部分 找到 texel

- ◆ A sample function samples the texel by dropping the fraction parts ($x=390$, $y=146$) and reading the texel colour values (e.g., $r=73$, $g=84$, $b=41$)
- ◆ A transform function maps the retrieved RGB values from the texture to ones that can be displayed, e.g., ($r=73$, $g=84$, $b=41$) $\rightarrow$ ($r'=73$, $g'=84$, $b'=41$) 有些渲染流程会对读出来的 RGB 再做一次处理
- ◆ For COMP3069, the RGB values are unchanged



1. 什么叫“放大”? (Magnification)

假设你有一张很小的纹理:

- 例如: 10×10 像素 (100 个 texel)

但你把它贴到屏幕上一个很大的物体上:

- 屏幕空间可能需要绘制 500×500 个像素 (25 万个 pixel)

纹理太小 $\rightarrow$ 只能用少量 texel 来填很多像素
于是:

多个屏幕像素只能重复使用同一个 texel

$\rightarrow$ 导致“马赛克”/“像素块”/“blocky”效果

2. 图中粉色方框表示什么?

图中:

- 白色小格子 = texel
- 黑色大格子 = 屏幕像素

粉色框示例说明:

- 粉框范围内有多个屏幕像素 (大格子)
- 但是纹理中对应的只有一个或极少数 texel (小格子)

因此:

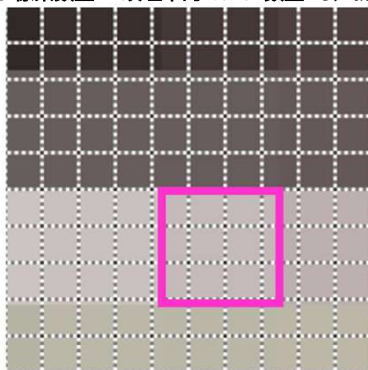
多个 pixel 只能 sample (采样) 同一个 texel

$\rightarrow$ 在视觉上看到的是“放大且模糊的大方块”

Magnification 放大

Magnification occurs during texture mapping when $pixels > texels$ which means that more than one pixel will sample the same texel

当屏幕上需要绘制的 像素数量 > 纹理中的 texel 数量 时, 就发生“放大现象” (Magnification)。



3. magnification 发生的数学条件

公式:

$$\text{if } \text{pixels} > \text{texels} \Rightarrow \text{magnification}$$

也可以理解为:

$$\frac{\text{pixel size}}{\text{texel size}} > 1$$

意思是:

- 一个 texel 被“放大”到覆盖多个 pixel。

4. 为什么放大会造成“方块化”?

因为使用 nearest sampling:

- texel 是离散单位
- 当纹理的细节不足时, 多个 fragment 会读同一个 texel
- 屏幕上就出现大块“大像素”效果

幻灯片:

UV = (0.4, 0.22) → tc = (102.40, 56.32)

说明这张纹理的大小是:

- width = 256
- height = 256

(老师用的是 256×256 的图, 所以 $0.4 \times 256 = 102.4$)

公式如下:

$$T_x = u \times \text{textureWidth}$$

$$T_y = v \times \text{textureHeight}$$

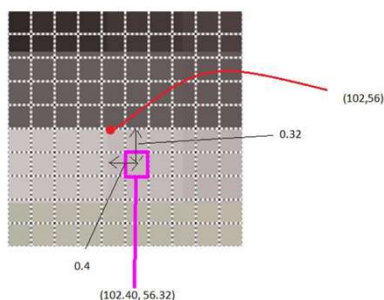
Mapping UV to Texel and Sampling

- ◆ The corresponder maps the fragment UV to a texel coordinate, e.g.,

$$\text{UV}=(0.4, 0.22) \rightarrow \text{tc}=(102.40, 56.32)$$

- ◆ The texel is sampled by dropping the fraction parts (*Nearest Sampling*) and reading the texel values

$$\text{tc}=(x=102, y=56) \rightarrow \text{col}=(r=73, g=84, b=41)$$



Magnification Example

- ◆ A polygon (made of triangles) 由三角形组成的多边形



Resulted Pixelated Magnification



用于双线性过滤的纹理元素

Texels for Bilinear Filtering

- ◆ *Bilinear filtering* is a method for smoothing the pixilation during magnification. 双线性过滤是一种在放大时平滑像素化的方法：当纹理被放大时，为了减少马赛克，我们不再只取一个 texel，而是取“周围 4 个 texel”并进行双线性插值，让颜色变得平滑。它可以显著减少 pixelated blocks。
- ◆ The *nearest 4 texels* are sampled, on either side of the texel coordinate including fraction parts,

$tc=(102.40, 56.32)$

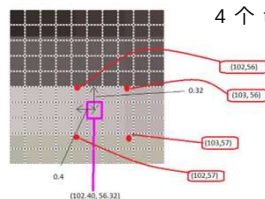
在纹理坐标（包括小数部分）的两侧，分别对最近的 4 个纹理元素进行采样

$tc1=(x=102, y=56) \rightarrow col=(r=73, g=84, b=41)$

$tc2=(x=103, y=56) \rightarrow col=(r=75, g=94, b=51)$

$tc3=(x=102, y=57) \rightarrow col=(r=105, g=94, b=31)$

$tc4=(x=103, y=57) \rightarrow col=(r=107, g=97, b=37)$



4 个 texel 的颜色各不相同，所以要通过插值来得到一个“平滑过渡”的颜色。

因此:

- $dx = 0.40$ (在 x 方向距离左边 texel 的比例)
- $dy = 0.32$ (在 y 方向距离下方 texel 的比例)

Step 1: 沿 x 方向插值 (左右方向)

插值公式:

$$c = (1 - dx) \cdot c_{\text{left}} + dx \cdot c_{\text{right}}$$

Step 2: 沿 y 方向插值 (上下方向)

现在我们有两行 x 方向插值后的颜色:

- 下方行: $C_{\text{lower}} = (73.8, 89.2, 45.0)$
- 上方行: $C_{\text{upper}} = (107.8, 95.2, 33.4)$

插值公式:

$$c = (1 - dy) \cdot C_{\text{lower}} + dy \cdot C_{\text{upper}}$$

Bilinear Filtering

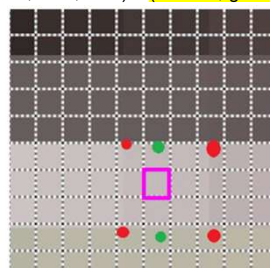
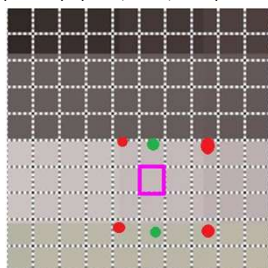
- ◆ Colours are interpolated in the x axis using the x fraction part $dx=0.4$

$$(1-0.4) \cdot (r=73, g=84, b=41) + 0.4 \cdot (r=75, g=94, b=51) = (73.8, 89.2, 45.0)$$

$$(1-0.4) \cdot (r=105, g=94, b=31) + 0.4 \cdot (r=107, g=97, b=37) = (107.8, 95.2, 33.4)$$

- ◆ Then, colour is interpolated in the y axis, using the y fraction part $dy=0.32$

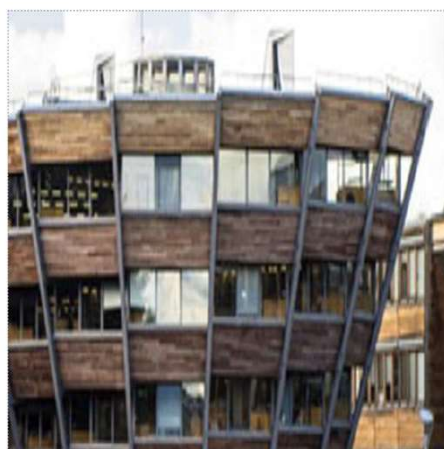
$$(1-0.32) \cdot (73.8, 89.2, 45.0) + 0.32 \cdot (107.8, 95.2, 33.4) = (r=84.68, g=91.12, b=41.29)$$



16

Results with Bilinear Filtering

- ◆ With bilinear filtering, the image isn't pixelated and the magnification is smoothed



17

Without/With Bilinear Filtering

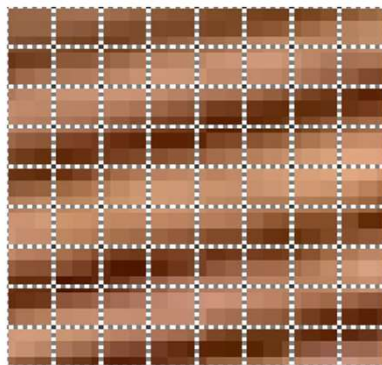


Minification & Mipmaps

当纹理元素 (texels) 数量多于像素 (pixels) 数量时, 就会发生纹理缩小现象, 因此一个片段会采样多个纹理元素的颜色值。

- ◆ Minification occurs when $\text{texels} > \text{pixels}$, so one fragment will sample multiple texel colour values
- ◆ In the figure, each pixel covers more than 2×2 texels

在图中, 每个像素覆盖了超过 2×2 个纹理元素。

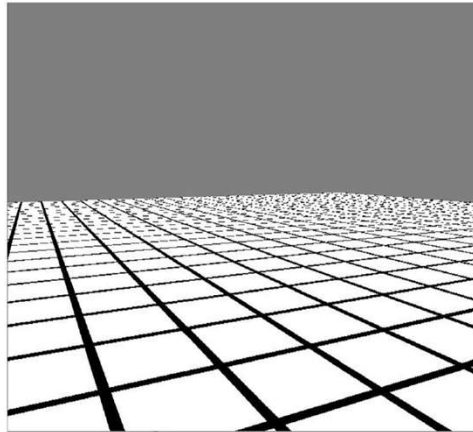


纹理缩小和混叠

Minification and Aliasing

在纹理采样过程中，每个片段仅会采样最近的纹理元素。

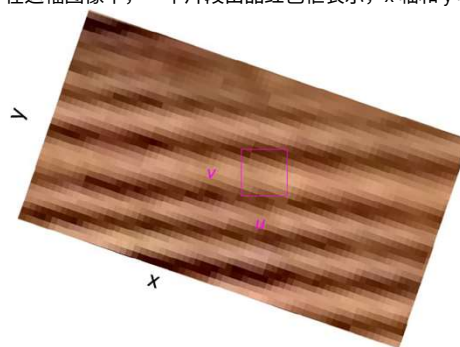
- ◆ During texture sampling, only the nearest texel will be sampled for each fragment



平均采样

Sampling by Averaging

- ◆ Sampling by averaging the colours and by calculating differentials 通过平均颜色以及计算差值来进行采样
- ◆ In this image, a fragment is represented by the magenta box, and the x and y axes are also sampled 在这幅图像中，一个片段由品红色框表示，x 轴和 y 轴也被采样了



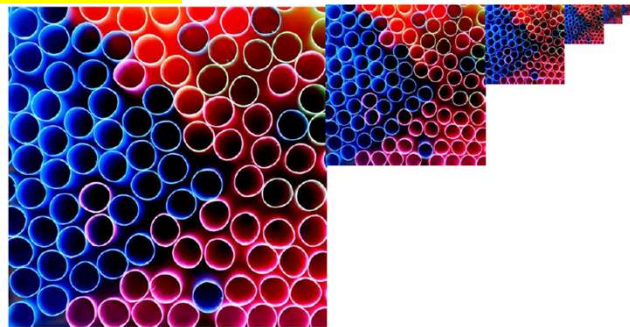
确定纹理数量

Determine the Number of Texels

- ◆ Calculate the change in x texels for one change in u texture coordinate $\partial x / \partial u = 7.2$ 计算纹理坐标 u 每变化一次 x 纹素的变化量 $l = 7.2$
- ◆ The change in y texels for one change in v texture coordinate $\partial y / \partial v = 7.2$ 计算纹理坐标 v 每变化一次 y 纹素的变化量 $l = 7.2$
- ◆ The change in x texels for one change in v texture coordinate $\partial x / \partial v = 2.4$ 计算纹理坐标 v 每变化一次 x 纹素的变化量 $l = 2.4$
- ◆ The change in y texels for one change in u texture coordinate $\partial y / \partial u = 2.4$ 计算纹理坐标 u 每变化一次 y 纹素的变化量 $l = 2.4$
- ◆ Then, take the maximum differential as the square size of the maximum number of texels covered by a fragment, $d = 7.2$ 然后, 取最大微分作为片段覆盖的最大纹素数量的正方形边长, $d = 7.2$

Mipmaps

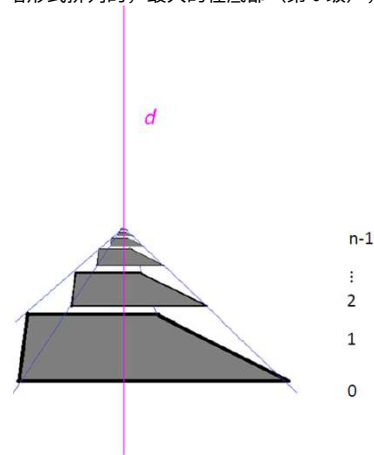
- ◆ Mipmaps are a way of storing multiple copies of the same image, each half the size of the previous image in the sequence Mipmaps 是一种存储同一图像多个副本的方法, 每个副本的尺寸都是前一个副本的一半。
- ◆ The sequence continues until the smallest image is 1x1 pixel in size 这个序列会一直持续到最小的图像尺寸为 1 × 1 像素为止。



Visualise Mipmaps

- Mipmaps are arranged in a pyramid with the largest at the base (level 0), and the smallest at the top (level $n-1$)
- d value is compared against the height in the mipmap pyramid
 $levela < \log_2 d < levelb$

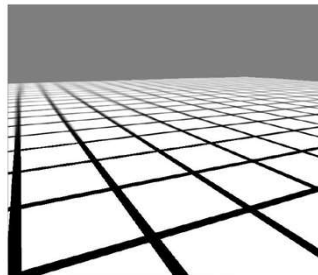
d 值与 mipmap 金字塔的高度进行比较



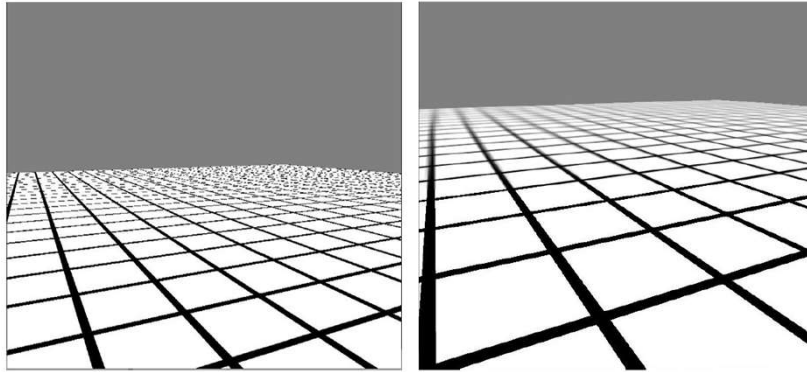
三线性采样

Trilinear Sampling

- In the example, $d = 7.2$, so $\log_2 7.2 = 2.85$, so levels 2 and 3 are used
- Bilinear sampling samples a colour c_a from level 2, and bilinear sampling samples a colour c_b from level 3 双线性采样从第 2 级采样颜色 c_a , 双线性采样从第 3 级采样颜色 c_b
- Then, bilinear sampling interpolates the colours c_a and c_b , to make the final pixel colour 双线性采样对颜色 c_a 和 c_b 进行插值, 以生成最终像素颜色



Without/With Trilinear Filtering



References

- ◆ *COMP3069 Lecture notes: Chapter 9 Textures*
- ◆ *Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL, Chapter 7*